Universidad Politecnica de Madrid

Explainable Classification of Payment Difficulty

Phases I–II: Decision Tree and Logistic Regression

Group members: Thales S. A. Camargo – thales.camargo@alumnos.upm.es
Jhon Corrales Almendariz – jhon.corrales@alumnos.upm.es
Joel Montes de Oca Martínez – joel.montesdeoca.martinez@alumnos.upm.es

Course: Explainable Artificial Intelligence
Programme: Master in Artificial Intelligence
Date: September 2026

Preliminary report for Tema 2 practical work

Abstract

This report compares a decision tree, logistic regression, and an Explainable Boosting Machine (EBM) for predicting next-month payment difficulty. All models use the same stratified hold-out split and training-only preprocessing; five-fold cross-validation guides model comparison and hyperparameter selection. On the hold-out split, the optimized EBM obtains 0.822 accuracy and 0.631 balanced accuracy, the tree 0.813 and 0.609, and tuned logistic regression 0.713 and 0.680, respectively. Thus, the EBM has the highest accuracy of these runs, while logistic regression has the highest balanced accuracy. Global explanations and local explanations for the same three validation instances are used to compare how the model families reach their predictions.

1 Introduction

The task is a binary classification problem based on customer credit and repayment information. The training data contain the target variable `payment_difficulty_next_month`; the test data contain the same predictors without that target. The objective is both predictive and explanatory: the model should produce test predictions while allowing its behavior to be inspected.

The decision-tree, logistic-regression, and EBM implementations are provided in the three phase notebooks under `code/`. All use a reproducible preprocessing pipeline, the same hold-out validation split, and model-specific prediction exports, while their model families and explanation methods remain distinct. The models favor class 0 to varying degrees: the tree and EBM retain higher overall accuracy, whereas tuned logistic regression gives the best balance between class recalls.

The models differ in how often they identify class 1, so balanced accuracy and class-specific recall are essential evidence rather than presenting accuracy in isolation. All three also provide different forms of explanation: explicit decision paths, additive linear coefficients, and learned additive feature effects.

2 Data and preprocessing

The notebook defines the feature roles explicitly rather than selecting columns implicitly. The identifier and target are excluded from the predictor matrix, while demographic, credit-limit, billing, payment, and repayment variables are used as predictors. Sex is retained as an audit attribute as well as a predictor, which makes later fairness-oriented analysis possible.

The categorical variables are sex, education, marital status, and repayment assistance plan. They are transformed with one-hot encoding using `handle_unknown="ignore"`, so categories that appear only in the test split do not cause prediction failures. Numerical variables are concatenated with the encoded categorical variables without additional scaling.

The labeled training data are split into training and validation partitions with a 70/30 ratio, a fixed random seed of 411, and stratification by the target. The unlabeled test data remain untouched until the final prediction stage. This procedure gives a reproducible validation estimate while preserving the test set for submission.

3 Method

The implementation uses pandas for tabular data, scikit-learn for splitting, encoding, cross-validation, parameter search, and evaluation metrics, and InterpretML for the class histogram, classification tree, logistic regression, EBM, and model explanations. Matplotlib is used for feature-importance, confusion-matrix, ROC, and predicted-class proportion plots. Shared data preparation and prediction-export functions are defined in `code/common.py`; dependencies are listed in `code/requirements.txt`.

Only the labeled training data are split. Seventy percent is used for fitting and thirty percent for validation, with stratification preserving the class proportions and `random_state=411` making the result reproducible. The separate test file is never used to fit the encoder or model: its categorical values are transformed after the encoder has been fitted on the training partition, with unknown categories ignored.

The common protocol begins with a 70/30 stratified split of the labeled data using `random_state=411`. Four categorical predictors are one-hot encoded by an encoder fitted only on the training partition; unknown categories are ignored, and numerical predictors are kept on their original scale. The same transformed training and validation rows are used for all three model families. Five-fold stratified cross-validation with seed 411 is run within the training partition. Balanced accuracy is the search objective because the target classes are imbalanced; accuracy and class-specific recalls are also reported. The hold-out validation partition is used for a common final comparison, while the unlabeled test data are excluded from fitting, tuning, and evaluation.

The tree search selects `max_depth=5` and `min_samples_leaf=50`; its search estimator is a class-weighted scikit-learn tree, while the selected structure is refit as an InterpretML `ClassificationTree`. Logistic regression selects $C = 100$ with balanced class weights. The EBM search selects `max_leaves=2` and `learning_rate=0.01`; it uses `interactions=0`, four outer bags, early stopping, and seed 411. Its zero-interaction constraint yields an additive model whose prediction is built from separate learned feature effects rather than pairwise interaction terms. Search scores are cross-validation selection scores, not unbiased estimates of the final refits; the same hold-out partition provides the directly comparable validation results.

4 Results

Table 1 begins the results with one row for each required estimator family. The cross-validation column reports the best balanced-accuracy search score; hold-out metrics are shown for context and are not substituted for the cross-validation comparison.

Table 1: Summary of the three fitted estimator families.

Classifier	Key settings	Five-fold CV balanced accuracy	Hold-out accuracy	H
Decision tree	depth 5; leaf size 50	0.690*	0.813	
Logistic regression	$C = 100$; balanced weights	0.671	0.713	
Explainable Boosting Machine	interactions=0; 2 leaves; learning rate 0.01	0.624	0.822	

*The tree search score comes from the class-weighted scikit-learn search estimator; the selected configuration is refit as an unweighted InterpretML tree. All search scores are used for selection and are not independent performance estimates.

4.1 Decision Tree

The hold-out tree accuracy is 0.813 and balanced accuracy is 0.609. Its class-0 recall is 0.96, compared with 0.26 for class 1. This gap accounts for the difference between the two aggregate metrics: the model detects most non-difficult cases but misses many difficult-payment cases.

The hold-out validation set contains 5,909 class-0 and 1,591 class-1 observations. No conclusion about test-set recall can be drawn because the test labels are unavailable.

The optimized tree's global explanation is generated with `tree.explain_global`. Figure 1 complements the interactive explanation with the fitted tree's split-based feature-importance ranking. Repayment status and the payment-to-bill ratio dominate the model's splits; this ranking describes the fitted tree and should not be read as a causal effect.

4.2 Logistic Regression

Table ?? summarizes the default and tuned logistic models. The default model nearly always predicts the majority class: its mean class-1 recall is 0.002 across the five folds, despite accuracy of 0.788. The grid search selects $C = 100$

Table 2: Decision-tree performance on the 30% stratified validation split.

Metric	Value
Accuracy	0.813
Balanced accuracy	0.609
Class 0 precision / recall / F1	0.83 / 0.96 / 0.89
Class 1 precision / recall / F1	0.65 / 0.26 / 0.37
Validation observations	7,500

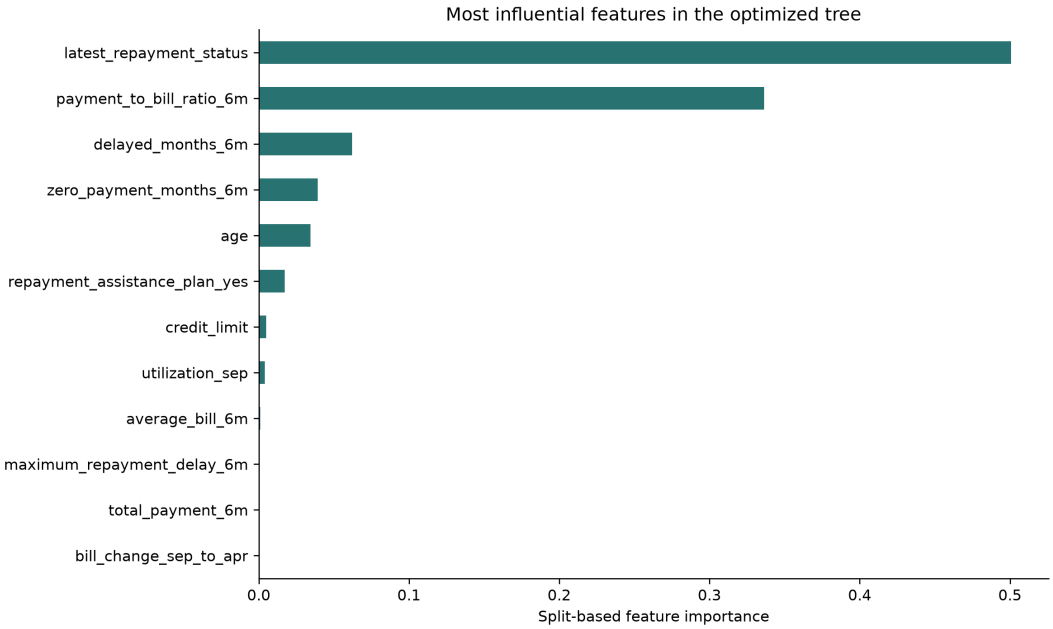


Figure 1: The twelve highest split-based feature importances for the optimized tree.

and balanced class weights. On the hold-out split, tuning raises balanced accuracy from 0.501 to 0.680, while accuracy falls from 0.788 to 0.713. This is a more balanced result than the optimized tree's 0.609 balanced accuracy, but the tree retains higher accuracy at 0.813. The two metrics therefore describe a genuine trade-off rather than one model dominating on every criterion.

The tuned model reaches 0.680 hold-out balanced accuracy, the strongest of the three, while its 0.713 accuracy is the lowest. Its default version predicts almost exclusively class 0 (five-fold class-1 recall 0.002 ± 0.002); class weighting increases sensitivity to the minority class at the cost of more false positives.

The optimized model's coefficient-magnitude ranking (Figure 2) places latest repayment status, zero-payment months, and delayed months among the strongest features. Repayment-assistance indicators and utilization also appear in the top twelve. The plot shows absolute coefficient magnitudes, so it indicates relative influence in this fitted linear model but not the direction of each association or a causal effect.

For the shared local cases used below, the logistic model assigns class-1 probabilities of 0.130, 0.656, and 0.377, respectively. With the 0.5 threshold, it correctly classifies the first two and misses the third; the local InterpretML explanation decomposes each prediction into feature contributions.

4.3 Explainable Boosting Machine

The EBM is a generalized additive model that learns a flexible, potentially nonlinear response curve for each feature and adds those effects to form the prediction. Setting `interactions=0` excludes pairwise terms, preserving a feature-wise explanation while allowing more flexibility than a linear logit. The search selects two leaves per feature and a learning rate of 0.01. On the hold-out set it reaches 0.822 accuracy and 0.631 balanced accuracy; recalls are 0.96 for class 0 and 0.30 for class 1. This gives the EBM the highest accuracy among these runs, but its class-1 recall and balanced accuracy remain below those of tuned logistic regression.

Figure 3 ranks the EBM's most influential additive terms by mean absolute contribution. Unlike the linear model's coefficient magnitudes, EBM term importance summarizes the magnitude of learned effects across observations; the direction and shape of each effect are available in the notebook's global explanation.

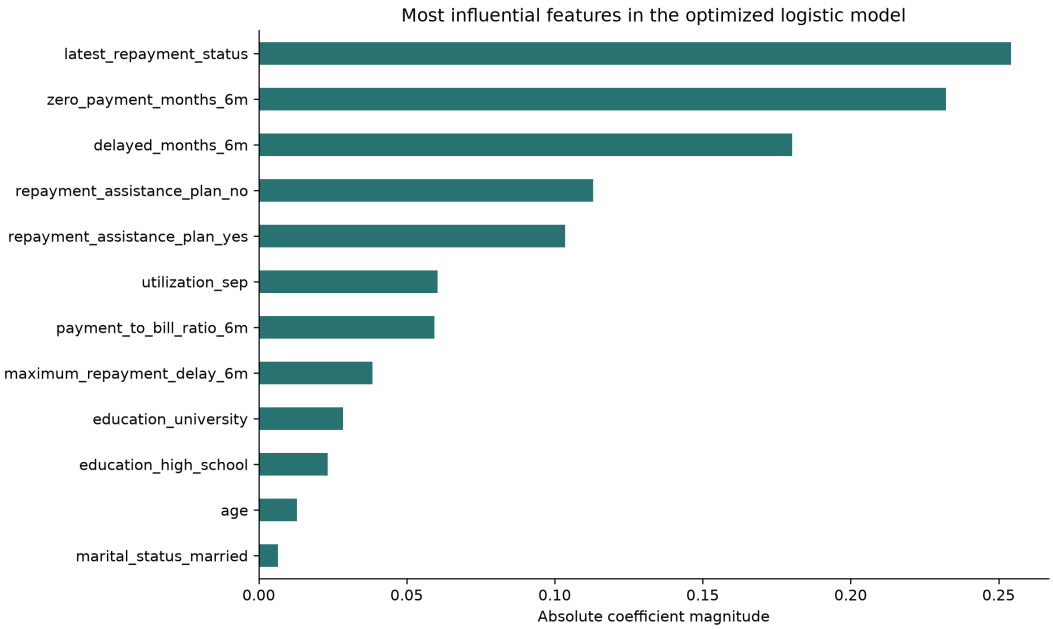


Figure 2: The twelve largest absolute coefficients in the optimized logistic model.

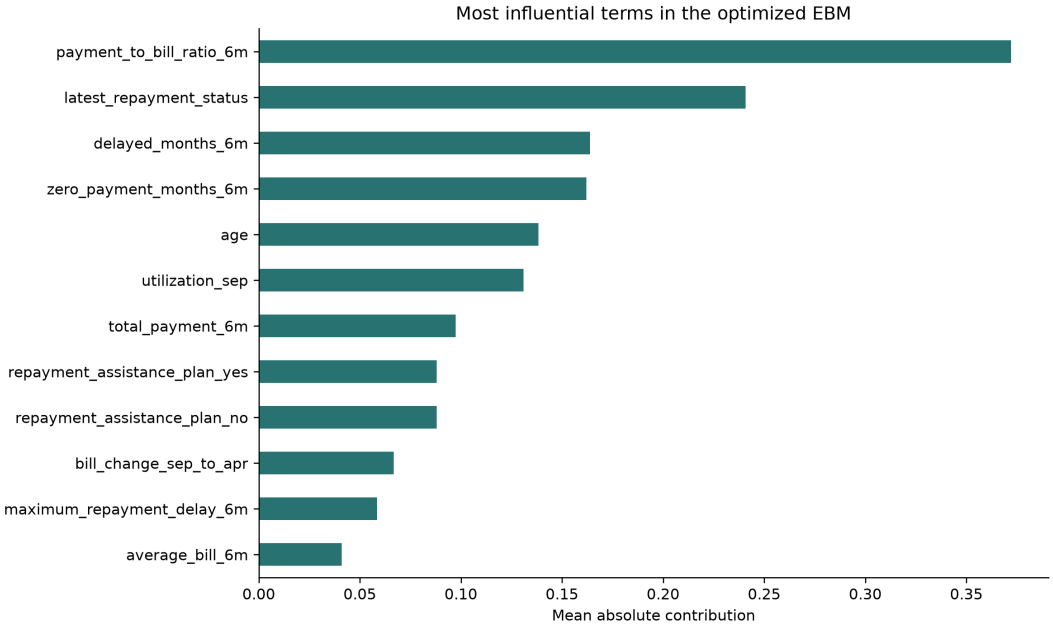


Figure 3: The twelve most influential additive terms in the optimized EBM.

5 Models

The classifier is `ClassificationTree` from the `interpret` package. A decision tree is suitable for this phase because its decision structure can be inspected directly and because the package exposes global explanations of the fitted model [Nori et al., 2019]. We used `GridSearchCV` to optimize hyperparameters, selecting a `max_depth` of 5 and `min_samples_leaf` of 50 based on balanced accuracy. The notebook also displays a class histogram for the training data, a confusion matrix, and a ROC curve with AUC metric.

The tree is transparent in principle: each prediction can be traced through a sequence of feature conditions, and the global explanation exposes the learned structure. Logistic regression offers a different form of transparency: its prediction is an additive function of the encoded features, and the coefficient magnitudes identify which terms most affect the fitted score. Neither representation should be interpreted causally, and the different explanation formats are not directly interchangeable.

The optimized tree's global explanation is generated with `tree.explain_global`. Its split-based feature-importance ranking places repayment status and the payment-to-bill ratio among the most influential features; this ranking describes the fitted tree and should not be read as a causal effect.

Figure 4 shows the first three levels of the optimized tree. The full depth-five tree is available in the notebook; limiting the report view keeps its main decision structure readable without hiding that lower-level branches exist.

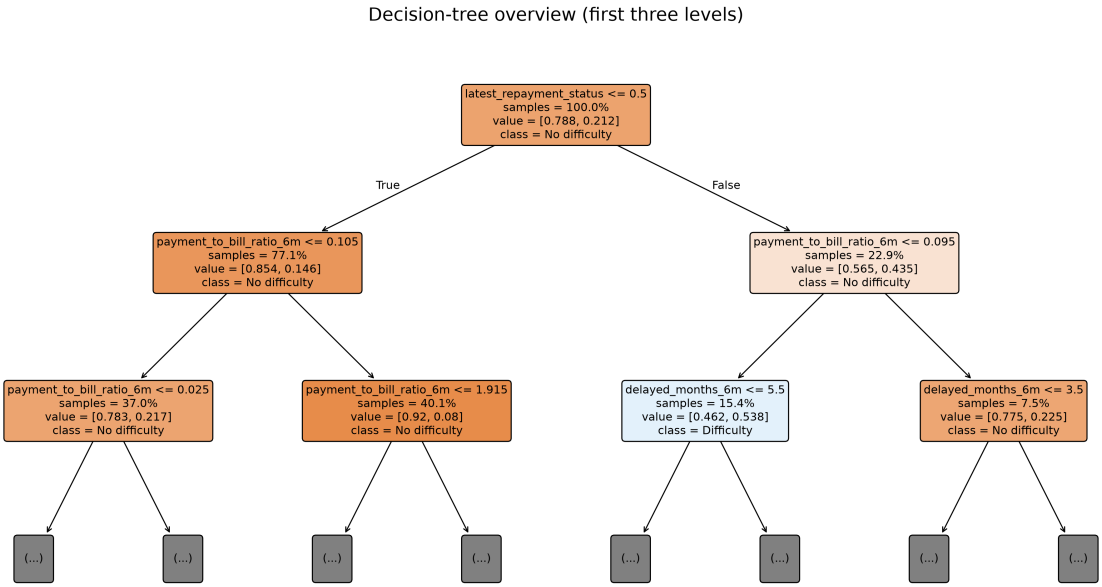


Figure 4: Readable overview of the optimized decision tree. Node colors indicate the predicted class and shading indicates class purity; lower branches are collapsed.

For local interpretation, the notebook selects fixed cases from the validation split, including correctly predicted examples from both classes and a missed class-1 example. Table 3 summarizes their paths. The missed positive reaches a class-0 leaf despite a repayment-assistance indicator, illustrating why the global model's strong majority-class behavior matters for individual decisions.

Table 3: Representative local decision paths from the validation set.

Case	Actual	Predicted	Path summary
Correct class 0	0	0	Non-delayed repayment status; payment-to-bill ratio above 0.105 and at most 1.91; no assistance plan; at most 2.5 zero-payment months.
Detected class 1	1	1	Delayed repayment status; payment-to-bill ratio at most 0.095; at most 5.5 delayed months; more than 2.5 zero-payment months; age above 25.5.
Missed class 1	1	0	Non-delayed repayment status; payment-to-bill ratio between 0.025 and 0.105; age above 24.5; assistance-plan flag is yes.

These are illustrative cases, not a substitute for evaluating the model across the full validation set. The interactive local explanations and exact split conditions for each selected case are included in the notebook.

The EBM's learned feature effects provide a compromise between the tree's discrete rules and the linear model's constant per-feature coefficients. With no interaction terms, each feature's contribution can be inspected separately, although interpreting the response curves still requires attention to their scales and direction.

6 Test predictions

Once validation is complete, each fitted model predicts the unlabeled test set. The notebooks preserve the original test predictors, append `predicted_class`, and save separate files: `data/decision_tree_test_predictions.csv` and `data/logistic_regression_test_predictions.csv`. Each also plots the predicted-class proportions. Keeping the original predictors beside the predictions makes the generated files easier to audit.

7 Limitations and future work

The current comparison includes a tree and logistic regression, but its grid-search scores are selection results and should not be treated as independent estimates. Future work should add the Explainable Boosting Machine, compare all three families under the same protocol, and assess threshold choices and class-specific error costs when false negatives and false positives are unequal. The tuned logistic results should also be supplemented with class-specific hold-out metrics when reporting the final model comparison.

The notebook already preserves sex as an audit attribute, but it does not yet calculate group-wise performance or fairness measures. A subsequent phase should compare error rates across relevant groups, inspect feature-importance stability, and use local explanations for the same representative instances across all three model families, including at least one instance from each class. Any such analysis should be performed without using the test labels.

8 Discussion

The tree is attractive because its rules can be communicated directly, while the logistic model provides an additive explanation through its coefficients. On the shared validation split, logistic regression achieves higher balanced accuracy than the tree after class-weighted tuning, but at lower accuracy. This reinforces the need to choose a model using the class-specific costs of errors rather than accuracy alone. Both tuned models improve on their majority-class-biased defaults, although the logistic model's default class-1 recall is almost zero.

Interpretability also has a cost. A fully expanded tree can become a long and repetitive list of rules, making individual paths technically transparent but difficult to understand. Limiting the depth to 5 not only improved the balanced accuracy but also restricted the tree size, making the global and local explanations much more understandable. The same trade-off should be assessed for logistic regression coefficients and additive feature-effect plots rather than assuming that one explanation format is universally superior.

9 Conclusion

The decision-tree and logistic-regression phases establish a reproducible pipeline from raw credit and repayment data to interpretable models and separate test prediction files. On the hold-out split, the tuned tree reaches 0.813 accuracy and 0.609 balanced accuracy, while tuned logistic regression reaches 0.713 accuracy and 0.680 balanced accuracy. The tree is more accurate overall; the logistic model is more balanced across classes. Adding the Explainable Boosting Machine and considering the cost of missed payment-difficulty cases are necessary before selecting a final model.

10 Use of generative AI

Generative AI was used to help organize the report structure, improve wording, and identify gaps between the Phase I notebook and the project statement. All numerical results and methodological claims in this report were checked against the repository notebook and its recorded outputs. The group remains responsible for the code, analysis, interpretation, and final submission.

References

Harsha Nori, Samuel Jenkins, Paul Koch, and Rich Caruana. Interpretml: A unified framework for machine learning interpretability. In *Proceedings of the 2019 Workshop on Human-Centric Machine Learning at NeurIPS*, 2019. URL https://github.com/interpretml/interpret.